

AI WeatherWise

Deliver Intelligent Weather Forecasts and Insights

AI-Powered Weather Backend Application

Node.js | Express.js | MongoDB | Google Gemini AI

JWT Authentication | Real-Time Weather Data | Personalized Recommendations

Project Documentation

Backend Architecture, Database Design & API Reference

INDEX

1. Description 3
2. Scenario-Based Case Study 3
3. Software & Hardware Requirements 4
4. Project Architecture (MVC Flow & Components) 4-5
5. Entity Relationship (ER) Diagram 6-7
6. Key Features 7
7. Roles and Responsibilities 7
8. User Flow 8
9. MVC Architecture Pattern 9
10. Project Setup and Configuration 10
11. Folder Structure & Environment Configuration 11
12. Backend Structure & Core Scripts 12
13. Database Configuration & Schema Definitions 13-14
14. Project Execution & API Testing 16-17
15. Demo Video & Conclusion 18

AI WeatherWise

Deliver intelligent weather forecasts and insights.

DESCRIPTION

AI WeatherWise is a robust RESTful backend application developed using Node.js, Express.js, and MongoDB. It provides real-time weather information and integrates Google Gemini AI to generate intelligent weather summaries and personalized recommendations. The system uses JWT authentication for secure user access and allows users to save and manage their favorite locations. It also includes centralized error handling and a fallback mode to ensure reliable API performance.

SCENARIO-BASED CASE STUDY

Background

Alex is a travel enthusiast who frequently visits different cities and needs a simple way to track weather conditions and receive AI-based activity suggestions.

Problem

- Difficulty managing multiple favorite weather locations.
- No personalized clothing or outdoor activity recommendations.
- Raw weather data is difficult for users to understand.
- User preferences need to be stored securely.

Solution

AI WeatherWise provides a centralized backend that retrieves real-time weather data, stores users and favorite locations in MongoDB, and uses Google Gemini AI to generate intelligent weather summaries and recommendations.

SOFTWARE REQUIREMENTS

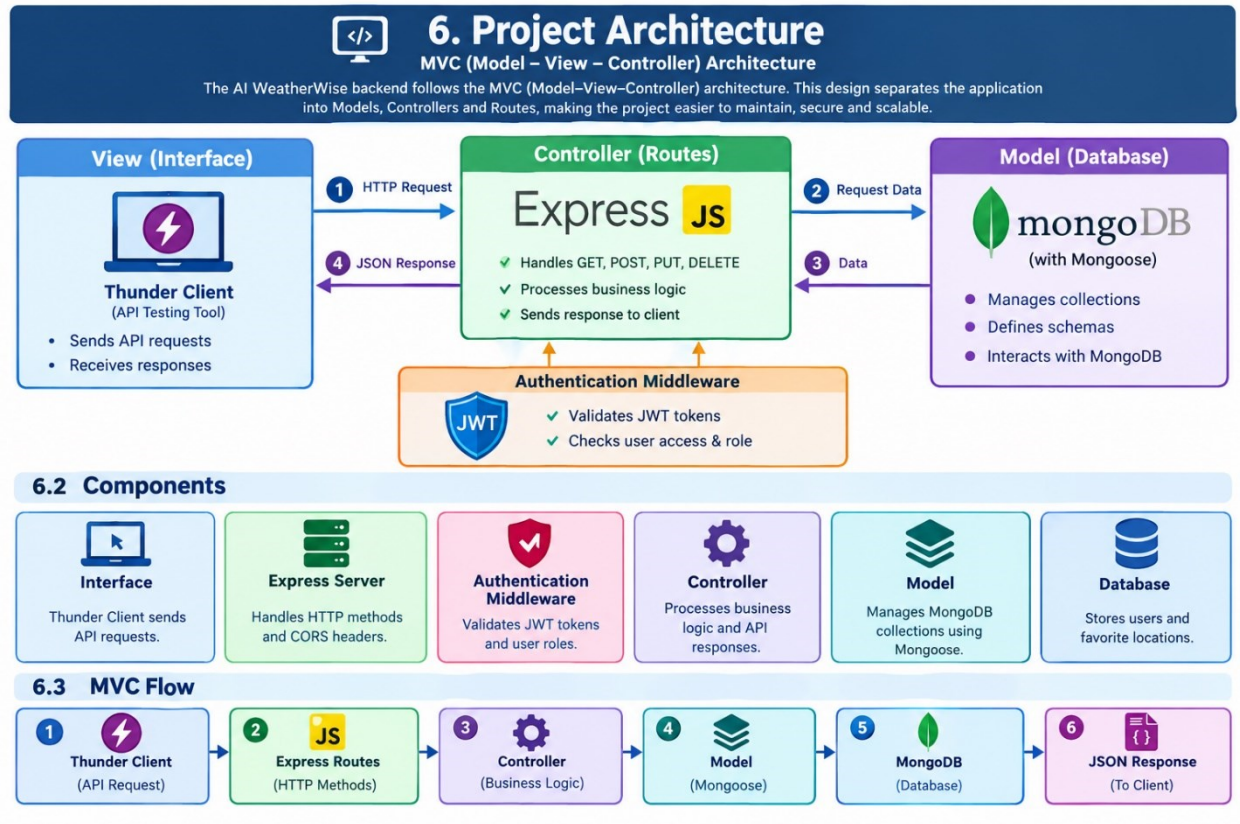
- **Operating System**: Windows 10/11, macOS, or Linux (supports cross-platform operations).
- **Node.js (v16 or above):** Runtime ecosystem running server logic and routing infrastructure.
- **npm (v8 or above):** Node package manager to control operational dependencies.
- **Gemini AI SDK:** Integration using @google/generative-ai for natural language insights.
- **MongoDB:** Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.
- **Postman:** API verification toolkit to assert schema validation responses across administrative route shields.
- **Code Editor:** Visual Studio Code or similar IDE environment.

Hardware Requirements

- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- **RAM:** 8 GB minimum (16 GB recommended for concurrent instances of MongoDB, processing layers, and execution nodes).
- **Storage:** 1 GB of available disk workspace.

PROJECT ARCHITECTURE

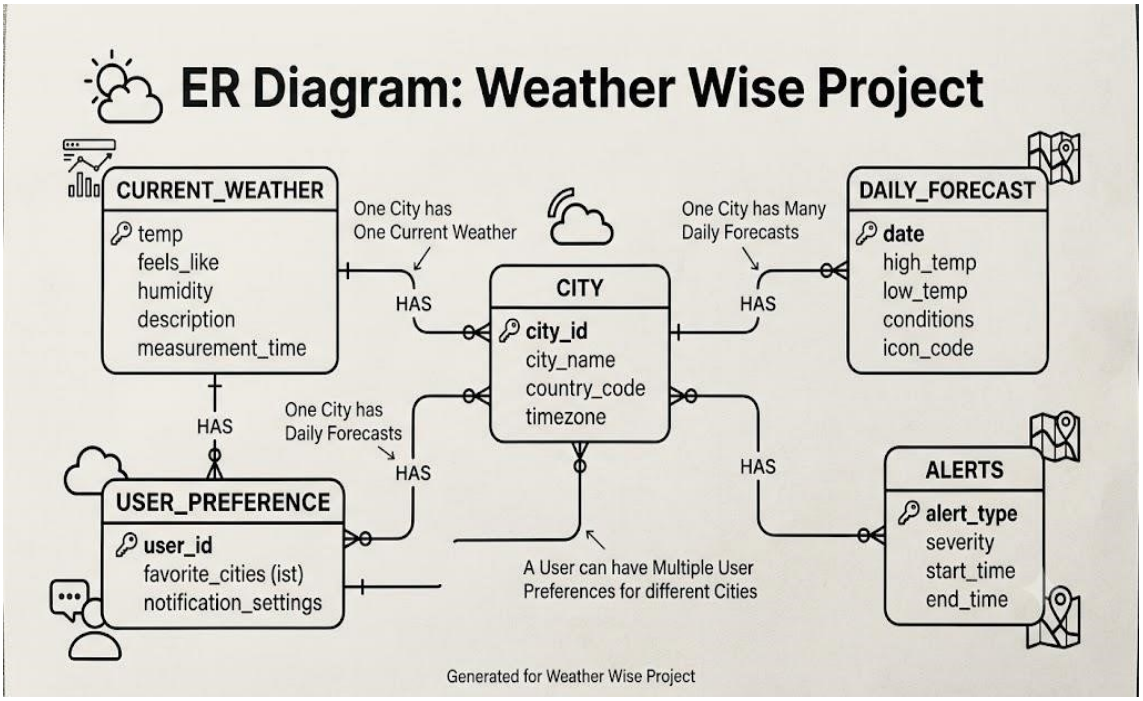
The AI WeatherWise backend follows the MVC (Model–View–Controller) architecture. This design separates the application into Models, Controllers, and Routes, making the project easier to maintain, secure, and scalable.



Components

- **Interface:** Thunder Client sends API requests.
- **Express Server:** Handles GET, POST, PUT, and DELETE requests.
- **Authentication Middleware:** Validates JWT tokens and user access.
- **Controller:** Processes business logic and API responses.
- **Model:** Manages MongoDB collections using Mongoose.
- **Database:** Stores users and favorite locations.

ENTITY RELATIONSHIP (ER) DIAGRAM



Entity & Attribute Breakdowns

1. USER

Stores registered user details and preferences.

- **user_id** (PK): Unique ID for each user.
- **username**: Display name of the user.
- **email**: User email address for authentication and notifications.
- **password_hash**: Encrypted password.
- **created_at**: Account creation timestamp.

2. LOCATION

Stores geographical coordinates and region details.

- **location_id** (PK): Unique ID for each location.
- **city_name**: Name of the city/town.
- **country**: Country code or name.
- **latitude**: Decimal latitude coordinate.
- **longitude**: Decimal longitude coordinate.

3. WEATHER_DATA

Stores current and historical weather logs fetched from external APIs or sensors for a specific location.

- **weather_id** (PK): Unique ID for the weather log entry.
- **location_id** (FK): Links to the LOCATION entity.
- **temperature**: Current temperature (e.g., in Celsius or Fahrenheit).
- **humidity**: Air moisture percentage.
- **wind_speed**: Wind velocity.
- **condition**: General summary (e.g., "Sunny", "Rainy", "Partly Cloudy").
- **timestamp**: Date and time when data was recorded.

KEY FEATURES

- **JWT Authentication** – Secure user login using JSON Web Token.
- **Real-Time Weather API** – Fetches current weather information.
- **AI Weather Summary** – Generates intelligent weather summaries using Gemini AI.
- **Personalized Recommendations** – Suggests activities based on weather conditions.
- **Favorite Locations** – Users can save and manage multiple cities.
- **Input Sanitization** – Protects the API from invalid or malicious requests.

ROLES AND RESPONSIBILITIES

Admin

- Manage user accounts.
- Monitor API health and system logs.
- Maintain database integrity.

Registered User

- Register and log in securely.
- Save and manage favorite locations.
- View real-time weather data.
- Get AI-based weather summaries and recommendations.
- Manage personal profile and credentials.

USER FLOW

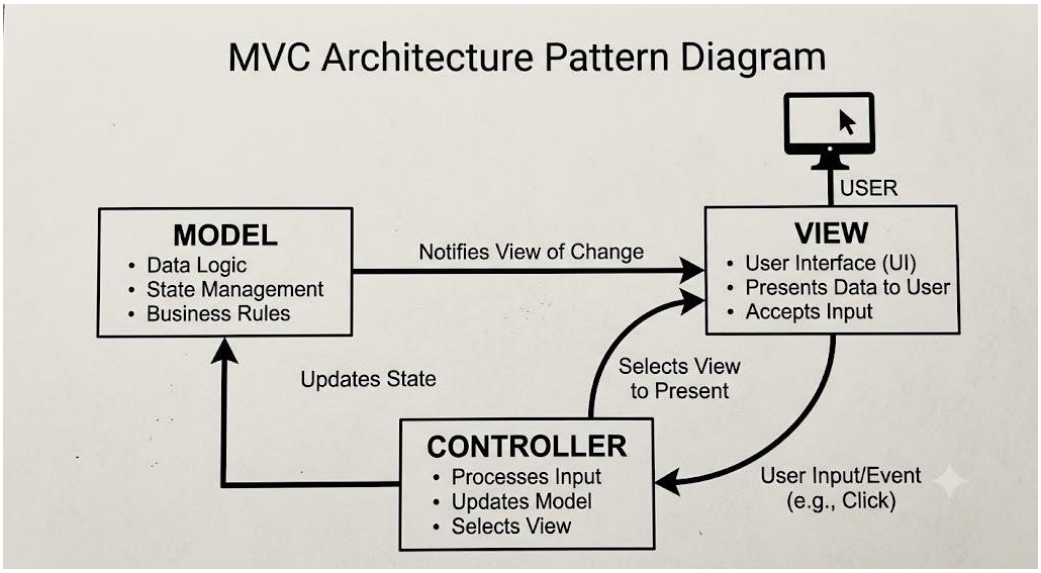
The following steps describe how a user interacts with the AI WeatherWise backend system.



Steps

1. User registers or logs in.
2. User saves a favorite city.
3. System fetches real-time weather data.
4. Gemini AI generates a weather summary and recommendation.
5. Backend returns the result as a JSON response to the user.

MVC Architecture Pattern



Model (Data & Logic)

- Manages the data, database interactions, business logic, and rules of the application.
- Notifies the View or Controller when data changes.
- Example: A User object fetching records from a database or processing password encryption.

View (User Interface)

- Handles the presentation layer and visual display of information to the user.
- Receives data from the Model or Controller and formats it (HTML, CSS, JSON, UI components).
- Does not perform data processing or business logic.
- Example: A webpage, mobile app screen, or dashboard displaying a user profile.

Controller (Brain & Coordinator)

- Acts as an intermediary between the Model and the View.
- Listens to user inputs or HTTP requests, updates the Model, and determines which View to display.
- Example: A login route handler that reads form inputs, validates user credentials using the Model, and redirects to the home View.

Project Setup and Configuration

Creating the Project Folder

- Create a new folder named AI Weatherwise
- Open the project folder in Visual Studio Code.
- Initialize the project using Node.js and install the required dependencies.
- Project Initialization

Open the project folder in the VS Code terminal and execute:

```
npm init -y
```

Install the required dependencies:

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv multer @google/genai
```

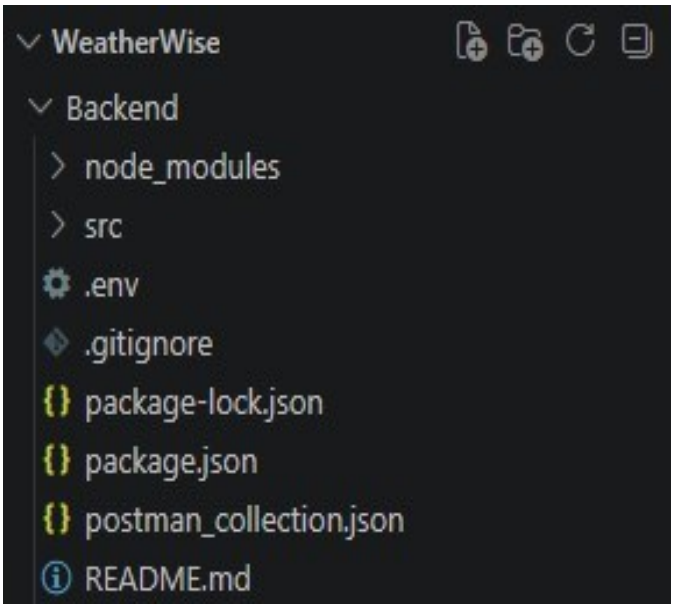
Install Nodemon for development:

```
npm install --save-dev nodemon
```

Create the Following Folder Structure

- index.js
- .env
- package.json
- .gitignore

Create the Following Folder Structure



Environment Configuration

Create a .env file in the project root and configure the required environment variables.

PORT=5000

MONGO_URI=your_mongodb_connection_string

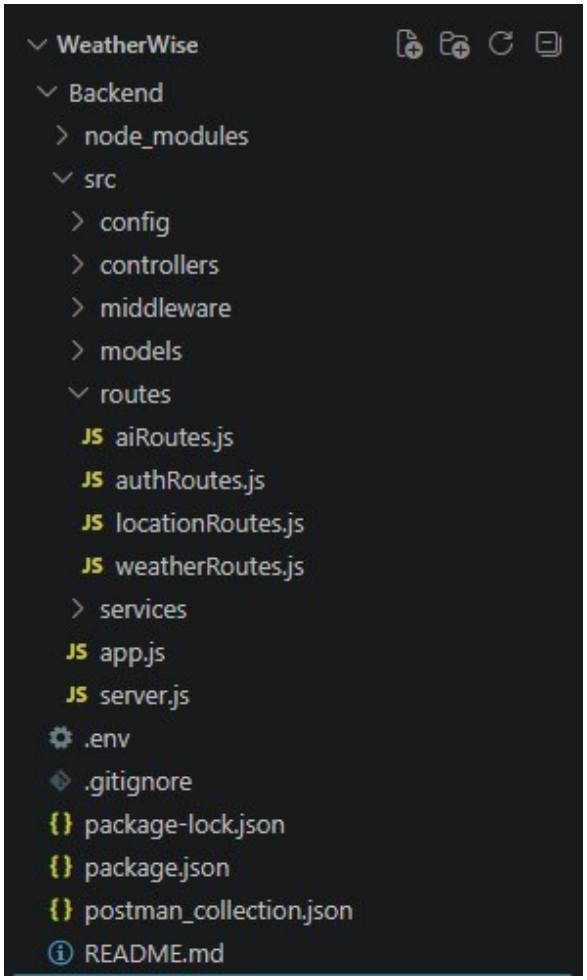
JWT_SECRET=your_jwt_secret

GEMINI_API_KEY=your_gemini_api_key

Backend Development

The AI weatherwise backend is developed using **Node.js**, **Express.js**, and **MongoDB**, following a modular architecture. The application separates routing, controllers, middleware, models, and utility modules to improve code maintainability and scalability. The backend exposes secure REST APIs for authentication, study material management, AI-powered content generation, and administrative operations.

Backend Structure:



Core System Script Implementations

Application Entry Point (index.js)

The **index.js** file serves as the main entry point of the application. It initializes the Express server, loads environment variables, configures middleware, establishes the MongoDB connection, registers API routes, and starts the server after a successful database connection.

Backend > src > JS server.js > ...

```
const app = require('./app');
const connectDB = require('./config/db');

const PORT = process.env.PORT || 5000;

connectDB()
  .then(() => {
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      // console.log('Press Ctrl+C to terminate.');
    });
  })
  .catch((error) => {
    console.error('Database connection failed. Express server aborted:', error.message);
    process.exit(1);
  });
```

Database Configuration API
Testing Demo :

1. Configure MongoDB:

- Install Mongoose.
- In your Node.js project directory, run:
- npm install mongoose • Create database connection.
- Create Schemas & Models.

Create Database connection:

Now, make sure the database is connected before performing any of the actions through the backend. The connection code looks similar to the one provided below

Mongoose Schema Definitions

Location Schema:



User schema:

Backend > src > models > JS User.js > ...

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters']
  }
}, {
  timestamps: true
});
```

Db connection:

This module establishes a connection between the application and MongoDB using Mongoose. It ensures that the backend communicates with the database before processing any client requests.

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/weatherwise');
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};
```

Project Executions:

The AI weatherwise backend can be executed locally using Node.js. Before running the application, ensure that MongoDB is running and the required environment variables are configured correctly. The backend exposes REST APIs that can be tested using Postman or Thunder Client.

Running the Backend Server

Install all required dependencies:

```
npm install
```

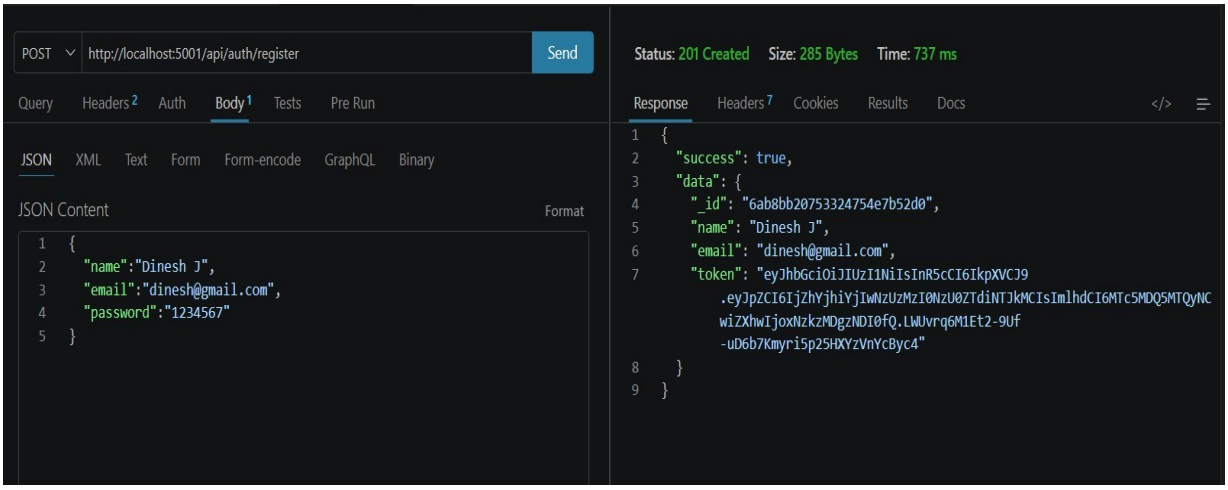
Start the development server:

```
npm start
```

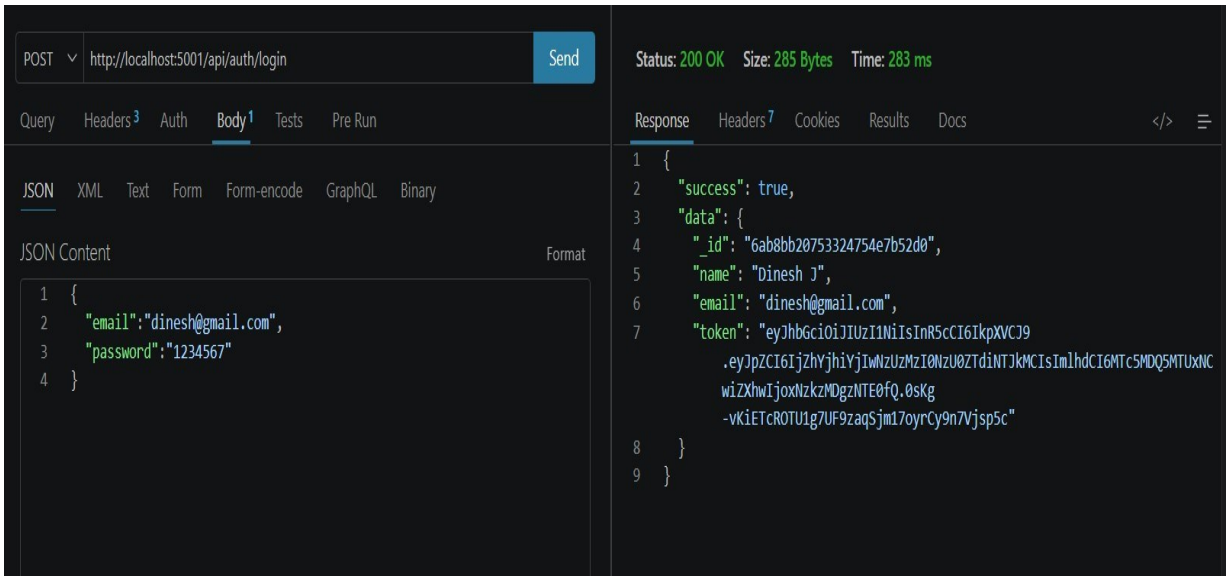
After successful execution, the server connects to MongoDB and starts listening for incoming API requests.

API testing:

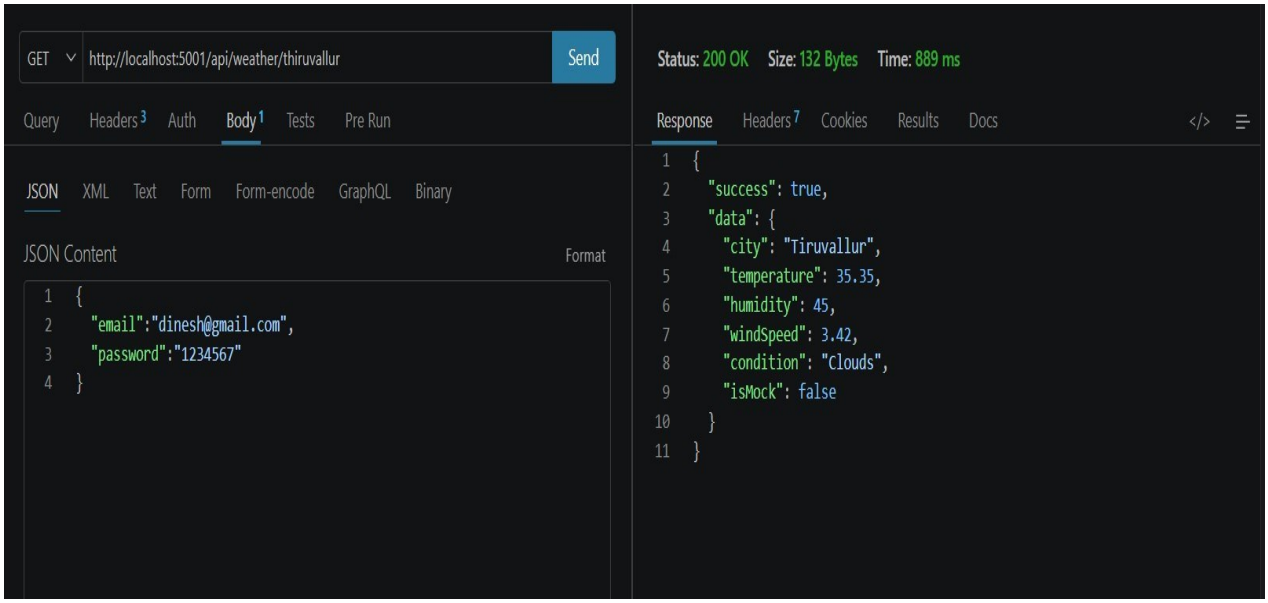
Register API:



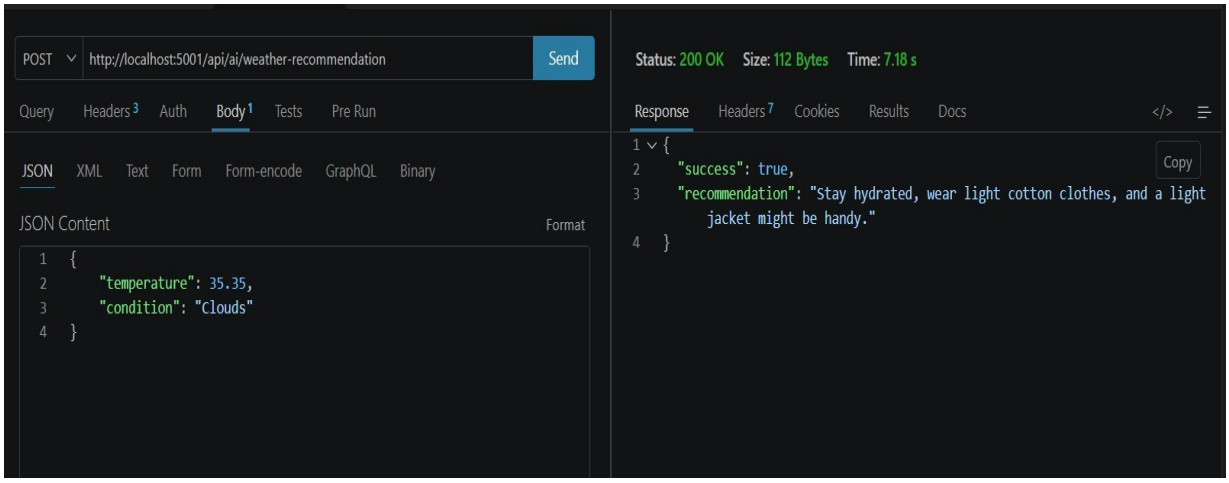
Login API:



Weather API:



Recommendation API:



Demo Video Link:

https://drive.google.com/file/d/1GVHQgL5Tsc0UtwMG1ZvbvqstkZGDmC9I/view?usp=drivesdk

THANK YOU